

١. هيكل المجلدات داخل الملف نفسة (Project Structure)

src/App.tsx	المكون الرئيسي الذي يحتوي على منطق التطبيق والواجهة.
src/main.tsx	نقطة الدخول الرئيسية للنظام (Entry Point).
src/index.css	ملف الأنماط ويحتوي على توجيهات Tailwind CSS.

ملاحظة: يمكن فصل المكونات لاحقاً إلى ملفات مستقلة (Components, Hooks, Types) لزيادة تنظيم الكود.

٢. الاستيرادات الأساسية (Imports)

TypeScript

```
import { useEffect, useRef, useState } from 'react';
import { Chart, ChartConfiguration, registerables } from 'chart.js';
import 'tailwindcss/tailwind.css';

// تسجيل مكتبة Chart.js للعمل في المشروع
Chart.register(...registerables);
```

شرح الأدوات المستخدمة:

- **Hooks (useState, useEffect, useRef):** لإدارة الحالة، تنفيذ العمليات الجانبية، والوصول لعناصر الـ DOM.
- **Chart.js:** المكتبة المسؤولة عن رسم المخططات البيانية.
- **Tailwind CSS:** لضمان ظهور التنسيق والجماليات البرمجية.

٣. مكون البطاقة الإحصائية (StatCard)

أولاً: تعريف الخصائص (Interface)

الخاصة	سيارة	اللون
title	string	عنوان البطاقة (مثل: إجمالي الطلبات).
value	string / number	القيمة الرقمية المعروضة.
trend	string (Optional)	نسبة التغير (اختياري).
icon	string	كلاس أيقونة Font Awesome.
borderColor	string	لون الإطار الجانبي.

ثانياً: كود المكون

TypeScript

```
const StatCard: React.FC<StatCardProps> = ({ title, value, trend, icon,
borderColor, iconBg, iconColor }) => (
  <div className={`bg-white rounded-2xl p-5 shadow-md border-r-4
    ${borderColor}`}>
    <div className="flex justify-between items-start">
      <div>
        <p className="text-gray-500 text-sm">{title}</p>
        <p className="text-3xl font-extrabold
          text-gray-800">{value}</p>
        {trend && <p className="text-xs mt-2"
          dangerouslySetInnerHTML={{ __html: trend }} />}
      </div>
      <div className={` ${iconBg} w-12 h-12 rounded-full flex
        items-center justify-center`}>
        <i className={` ${icon} ${iconColor} text-2xl`} ></i>
      </div>
    </div>
  </div>
);
```

٤. مكون الرسم البياني الدائري (PieChart)

هذا الجزء مخصص لشرح كيفية عرض نسب الامتثال والمخالفات.

شرح منطق العمل (للمطورين):

١. **useRef**: يستخدم لربط عنصر الـ canvas برمجيًا.
٢. **useEffect**: يضمن تحديث الرسم البياني فور تغير البيانات القادمة من الـ API.
٣. **Destroy**: يتم تدمير النسخة القديمة من الرسم قبل رسم الجديدة لمنع تداخل الرسوم وتوفير الذاكرة.

الميزانية	الحم
Type	pie (دائري).
Responsive	true (ليتجاوب مع مقاسات الشاشات المختلفة).
Legend	تم وضعه في الأسفل (Bottom) مع استخدام خط "Cairo".

٥. مكون الرسم البياني الخطي (LineChart)

يستخدم لتتبع "عدد المخالفات الأسبوعية".

أهم الخصائص البرمجية:

- **Tension (٠,٣)**: يعطي انحناءً انسيابياً للخط بدلاً من الزوايا الحادة.

Eng/Raghad Waleed

- **Fill (true):** يقوم بتلوين المساحة تحت الخط بلون شفاف لزيادة الجمالية.
- **Point Radius:** تم ضبط حجم النقاط لتسهيل القراءة عند تمرير الماوس.

٦. جدول الطلبات المالية (Requests Table)

تستخدم المصفوفة `requestsData` لعرض البيانات في جدول منظم.

أنواع الحالات (Status Types):

١. **Compliant:** تظهر بخلفية خضراء (متوافق).
٢. **Violation:** تظهر بخلفية حمراء (مخالفة).
٣. **Pending:** تظهر بخلفية صفراء (قيد المراجعة).

٧. ملاحظات لتحسين الأداء والنظافة

١. **فصل المكونات:** يفضل وضع كل مكون في ملف مستقل لسهولة الصيانة.
٢. **إدارة الحالة:** عند توسع المشروع، يفضل استخدام (Context API) أو (Zustand).
٣. **التعامل مع الأخطاء:** يجب إضافة `try/catch` عند جلب البيانات من الخلفية (Backend).
٤. **الوصول (Accessibility):** التأكد من إضافة `aria-labels` للأزرار لتسهيل استخدام النظام لذوي الاحتياجات الخاصة.

٨. خلاصة الربط مع الـ API

لتحويل النظام من بيانات ثابتة إلى بيانات حقيقية، نتبع الخطوات التالية:

- استخدام `useState` لتعريف مصفوفات فارغة للبيانات.
- استخدام `fetch` أو `axios` داخل `useEffect` لجلب البيانات عند تحميل الصفحة.
- تحديث الحالة (State) بالبيانات الجديدة، مما سيؤدي لتحديث الرسوم البيانية والجداول تلقائياً.